Adversarial Driving: Attacking End-to-End Autonomous Driving

1st Han Wu
Computer Science
The University of Exeter
Exeter, the United Kingdom
hw630@exeter.ac.uk

2nd Syed Yunas
Computer Science
The University of the West of England
Bristol, the United Kingdom
syed.yunas@uwe.ac.uk

3rd Sareh Rowlands
Computer Science
The University of Exeter
Exeter, the United Kingdom
s.rowlands@exeter.ac.uk

4th Wenjie Ruan
Computer Science
The University of Exeter
Exeter, the United Kingdom
w.ruan@exeter.ac.uk

5th Johan Wahlström*
Computer Science
The University of Exeter
Exeter, the United Kingdom
j.wahlstrom@exeter.ac.uk

2023 IEEE Intelligent Vehicles Symposium (IV) | 979-8-3503-4691-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/IV55152.2023.10186386

***Abstract*—As research in deep neural networks advances, deep convolutional networks become promising for autonomous driving tasks. In particular, there is an emerging trend of employing end-to-end neural network models for autonomous driving. However, previous research has shown that deep neural network classifiers are vulnerable to adversarial attacks. While for regression tasks, the effect of adversarial attacks is not as well understood. In this research, we devise two white-box targeted attacks against end-to-end autonomous driving models. Our attacks manipulate the behavior of the autonomous driving system by perturbing the input image. In an average of 800 attacks with the same attack strength (epsilon=1), the image-specific and image-agnostic attack deviates the steering angle from the original output by 0.478 and 0.111, respectively, which is much stronger than random noises that only perturbs the steering angle by 0.002 (The steering angle ranges from [-1, 1]). Both attacks can be initiated in real-time on CPUs without employing GPUs. Demo video: https://youtu.be/I0i8uN2oOP0.**

***Index Terms*—Adversarial Attacks, Imitation Learning, Deep Neural Network.**

I. Introduction

Autonomous driving is one of the most challenging tasks in safety-critical robotic applications. Most real-world autonomous vehicles employ modular systems that divide the driving task into smaller subtasks. In addition to a perception module that relies on deep learning models to locate and classify objects in the environment, modular systems also include localization, prediction, planning, and control modules. However, researchers are also exploring the potential of end-to-end driving systems. An end-to-end driving system is a monolithic module that directly maps the input to the output, often using deep neural networks. For example, the NVIDIA end-to-end driving model [1] maps raw pixels from the front-facing camera to steering commands. The development of end-to-end driving systems has been facilitated by recent advances in high-performance GPUs and the development of photo-realistic driving simulators, such as the Carla Simulator [2] and the Microsoft Airsim Simulator [3].

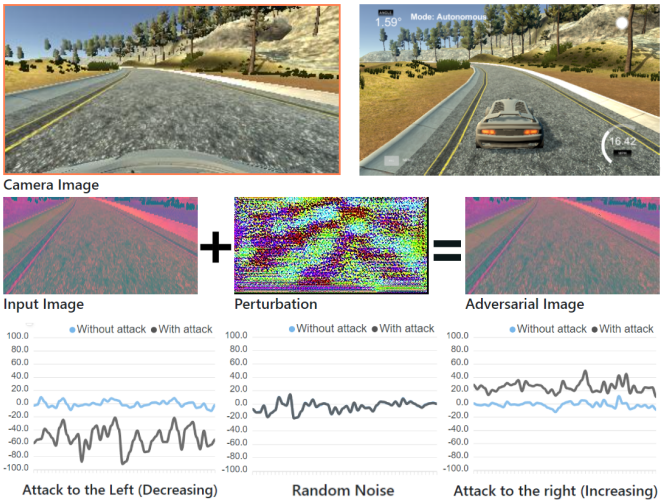


Fig. 1: Adversarial Driving: The behavior of and end-to-end autonomous driving model can be manipulated by adding unperceivable perturbations to the input image.

As demonstrated in multiple contexts, deep neural networks are vulnerable to adversarial attacks. Typically, these attacks fool an image classification model by adding an unperceivable perturbation to the input image [4]. Despite the fact that the number of academic publications discussing end-to-end deep learning models is steadily increasing, their safety in real-world scenarios is still unclear. Though end-to-end models may lead to better performance and smaller systems, the monolithic module is also particularly vulnerable to adversarial attacks. In addition, note that current research on ad-

The project is supported by Offshore Robotics for Certification of Assets (ORCA) Partnership Resource Fund (PRF) on Towards the Accountable and Explainable Learning-enabled Autonomous Robotic Systems (AELARS) [EP/R026173/1]. .

versarial attacks primarily focuses on classification tasks. The effect of these attacks on regression tasks, however, largely remains unexplored. Our research explores the possibility of achieving real-time attacks against NVIDIA's end-to-end regression model (See Fig. 1). However, the attacks may also be applied to the perception module in a modular driving system.

The main contributions of this paper are as follows:

- We propose two online white-box adversarial attacks against an end-to-end regression model for autonomous driving: one strong attack that generates the perturbation for each frame (image-specific), and one stealth attack that produces a universal adversarial perturbation that attacks all frames (image-agnostic).
- The robustness of the attacks is illustrated using experiments conducted in Udacity Simulator. The experiments demonstrate that it only takes the attack a few seconds to deviate the vehicle to outside of the lane.
- To facilitate future extensions and benchmark comparisons, our attack is open-sourced on Github[1]. As far as the authors are aware, this is the first open-source real-time attack on regressional driving models.

II. Preliminaries

This section categorizes and describes end-to-end driving systems and associated adversarial attacks.

A. End-to-End Driving Systems

End-to-end driving systems treat the driving pipeline as a monolithic module that maps sensor inputs directly to steering commands [5]. Typically, end-to-end driving systems are implemented using either imitation learning or reinforcement learning. Imitation learning methods use deep neural networks to learn and mimic human driving behavior [6]. A supervisor is responsible for feeding the algorithm with labeled data. Reinforcement learning methods, on the other hand, improve driving policies via exploration and exploitation. The training process is not dependent on the existence of any supervisor. While there is a growing trend of publications that use reinforcement learning [7] [8] [9] [10] [11], imitation learning is still more popular in end-to-end driving models [12] [13] [14] [15]. For this reason, our research will also focus on attacking imitation learning models.

The first implementation of an imitation-learning-based end-to-end driving system was the Autonomous Land Vehicle in a Neural Network (ALVINN) system, which trained a 3-layer fully connected network to steer a vehicle on public roads [16]. However, end-to-end driving models have also been applied for the task of off-road driving [17]. More recently, researchers from NVIDIA built a convolutional neural network to map raw pixels from a single front-facing camera directly to steering commands [1]. The NVIDIA end-to-end driving model is the target model in this paper, and details on this model are presented in Section III.

[1]The code is available on Github: https://github.com/wuhanstudio/adversarial-driving

B. Adversarial Attacks

This paper will consider an end-to-end driving model that outputs continuous steering commands, which is a regression model. Prior research on adversarial attacks primarily focuses on attacking classification models [18] [19] [20] [21].

A successful attack against classification models deviates the output from the correct label. Taking the digital handwritten digit classification task as an example, an attacker can fool the classifier into recognizing the number 3 as 7. To evaluate the performance of an adversarial attack against a regression model, we need to quantify the magnitude of the resulting deviation. An attack that causes the steering angle to deviate from 1.00 to 0.99 will typically be considered unsuccessful since such a tiny deviation may not have any noticeable effect on the driving outcome. To be considered successful, an attack must lead to larger deviations. Prior research used Root Mean Squared Error (RMSE) [22] and Mean Square Error (MSE) [23] to evaluate and compare deviations. A successful attack should produce a higher MSE or RMSE than random attacks. Boloor et al. attacked an end-to-end self-driving model using human-perceivable physical shadow [24], while our research focuses on generating human-unperceivable perturbations.

While prior research primarily focuses on offline attacks against classification models, we investigate online attacks against regression models. Offline attacks apply perturbations to static images. Under the scenario of autonomous driving, an offline attack splits the driving record into static images and the corresponding steering angles. The perturbation is then applied to each static image, and the attack is evaluated using the overall success rate [25]. Online attacks, on the other hand, apply the perturbation in a dynamic environment. Rather than applying the perturbation to static images in a driving record, we deploy the perturbation while the vehicle is driving. This also makes it possible to investigate the driving models' reactions to the attacks.

One big difference between online and offline attacks is that the ground truth is unavailable in online attacks. Offline attacks take pre-recorded human drivers' steering angles as the ground truth, while real-time online attacks do not have access to pre-recorded human decisions. Therefore, we use the model output under normal benign conditions as the ground truth and assume that the driving model is comparatively close to the ground truth. This assumption is reasonable since if the model is inaccurate, the erroneous model is already a threat in itself. There is no need to attack the system in the first place.

Existing adversarial attacks can be categorized into white-box, gray-box, and black-box attacks [26]: In white-box attacks, the adversaries have full knowledge of the target model, including model architecture and parameters; In gray-box attacks, the adversaries have partial information about the target model; In black-box attacks, the adversaries can only gather information about the model through querying. Since white-box attacks are more efficient, we devise two white-box attacks that achieve real-time performance against end-to-end driving models .

III. Problem Formulation

In this section, we specify our objective, introduce mathematical notation, and describe our target model. Throughout the paper, we will use the notation

$$y = f(\theta, x) \tag{1}$$
$$y' = f(\theta, x') \tag{2}$$

where y is the benign output steering command, $f(\theta, x)$ is the regression model that maps input images to steering commands, θ is the model parameters, x is the original input image, y' is the adversarial output steering command, and x' is the adversarial input image. Further, we will use $\eta = x - x'$ to denote the adversarial perturbation, y^* is the ground truth steering command, and $J(y, y^*)$ to denote the training loss. Given an input image x, the objective of attacking a classifier is to generate a small perturbation η, such that $y' \neq y^*$. However, the objective of attacking a regression model is to generate a small perturbation η, such that the difference between y' and y^* is larger than the average deviation caused by adding random noise to x.

We use the L_2 norm to quantify the magnitude of the perturbation. The L_2 norm of the perturbation η should be smaller than 0.03 (8 / 255) for an RGB input image according to the value used in prior research [27] [28]. In particular, to ensure that the perturbation is unperceivable to human eyes, we require

$$||x^{'} - x||_2 = || \ \eta \ ||_2 \leq \xi \tag{3}$$

where $\xi = 0.03$.

Our target model is the NVIDIA end-to-end driving model [1]. The input shape of the model is (160, 320, 3), which represents (height, width, channel) respectively. The output steering angle is in the range of $[-1, 1]$ on all our (unperturbed) collected images. An output of -1 represents steering to the left, and an output of 1 means steering to the right. The input image is captured by the front camera, and we then apply predefined preprocessing methods before feeding the image to the model. Refer to [1] for details on these preprocessing methods, including cropping, resizing, and RGB to YUV.

IV. Adversarial Attacks

In this section, we devise two white-box attacks against the driving system: one image-specific attack and one image-agnostic attack. Then, we present the system architecture.

A. Image-specific Attack

The first adversarial attack against a classifier was an image-specific offline attack that generated one perturbation for every input image [4]. Instead of minimizing the training loss $J(y,\ y^*)$, Goodfellow et al. maximized the training loss and then used the gradient of the training loss to generate the perturbation. However, online attacks do not have access to the ground truth y^*, and thus, for online attacks, the training loss $J(y, y^*)$ cannot be calculated. As a result, we need a new adversarial loss $J(y)$ that only requires the model output y to generate the perturbation.

Algorithm 1 Image-specific Attack

Input: The regression model $f(\theta, x)$, the input images $\{x_t\}$ where x_t is the image at time step t.
Parameters: The strength of the attack ϵ.
Output: Image-specific perturbation η.
for each time step t **do**
  Inference: $y = f(\theta, x)$
  Perturbation: $\eta = \epsilon \ \text{sign}[\nabla_x(J(y))]$
end for

When attacking a regression model, notice that we have the choice to either increase or decrease the output. For example, to attack the end-to-end driving regression model, we can either deviate the vehicle to the left by decreasing the output or to the right by increasing the output. Therefore, in some sense, attacks on regression models can be seen as a special case of attacks on classification models, with the constraint that we only have two choices: increasing or decreasing the output. Accordingly, we will consider the straightforward adversarial loss functions

$$J_{\texttt{left}}(y) = -y \tag{4}$$
$$J_{\texttt{right}}(y) = y \tag{5}$$

for the image-specific attack.

As explained in Section II-B, the adversarial loss functions $J(y)$ do not include ground truth y^*, which we do not have access to for online attacks. We can then utilize the Fast Gradient Sign Method (FGSM) to generate perturbations as

$$\eta = \epsilon \ \text{sign}[\nabla_x(J(y))] \tag{6}$$

where ϵ is a scaling factor that determines the visibility of the perturbation. The image-specific attack is summarized in Algorithm 1.

As an example, assume that the attacker wishes to attack the vehicle to the right side. In this case, the objective is to increase the model output. We can then use the adversarial loss $J_{right}(y)$ to generate the perturbation. $\nabla_x(J(y))$ represents the gradient of the adversarial loss over the input. The gradient gives us information regarding how changes in the adversarial loss y will back-propagate to the input.

B. Image-agnostic Attack

Even small deviations may cause traffic accidents. A small deviation in the steering angle may, for example, result in a failure to steer around a sharp corner. In other words, even if the attack is not as strong as the image-specific attack it could still be perilous if applied at critical time points. Therefore, we introduce a white-box attack that generates a universal adversarial perturbation (UAP) [29] which can be used to attack all input images at different time steps. The image-agnostic attack combines the idea of DeepFool [30] and Projected Gradient Descent (PGD) [31]. The attack consists of two procedures: training and deployment. We first generate a UAP online or via a driving record and then deploy the UAP.

Algorithm 2 Image-agnostic Attack (Training)

Input: The regression model $f(\theta, x)$, input images in a driving record X, the target direction $I \in \{-1, 1\}$.
Parameters: the number of iterations n, the learning rate α, the step size ξ , and the strength of the attack ϵ measured by the l_∞ norm.
Output: Image-agnostic perturbation η.
Initialization: $\eta \leftarrow 0$
for each iteration **do**
  for each input image x in the driving record X **do**
    Inference: $y = f(\theta, x + \eta)$
    if sign$(y) \neq I$ **then**
      $x^{'} = x + \eta$
      $\eta_t \leftarrow 0$
      while sign$(y) \neq I$ **do**
        Gradients: $\nabla = \frac{\partial J(y)}{\partial x'}$
        Perturbation: $\eta_t = \eta_t + proj_2(\nabla,\ \xi)$
        Inference: $y = f(\theta, x + \eta_t)$
      end while
      $\eta = proj_\infty(\eta + \frac{\alpha}{\xi}\eta_t,\ \epsilon)$
    end if
  end for
end for

We first decide our target direction, that is, whether to attack the vehicle to the left ($y < 0$) or to the right ($y > 0$), and then choose the corresponding adversarial loss function ($J_{\texttt{left}}(y)$ or $J_{\texttt{right}}(y)$). The perturbation is initialized as zero. For each input image at each timestep, if the direction of the model output is not the same as the desired direction, we find the minimum perturbation that changes the sign of the model output to the desired direction.

To change the direction of the model output with the minimum perturbation, we calculate the gradient of the adversarial loss $J(y)$ and then project the gradient to the L_2 ball. The closed-form solution to the optimization problem $\arg\min\ ||\ \eta - \eta^{'}\ ||_2$ with the constraint $||\ \eta^{'}\ || \leq \xi$ is given by

$$proj_2(\eta,\ \xi) = \frac{\eta}{\max\{1, \frac{||\ \eta\ ||}{\xi}\}} = \eta \min\{1, \frac{\xi}{||\ \eta\ ||}\} \quad (7)$$

which can be proved using the Lagrangian and the KKT conditions [32].

After applying the temporary perturbation η_t at timestep t, if the direction of the model output matches the desired direction, we incorporate the temporary perturbation η_t to the overall perturbation η and then project η on the l_2 ball centred at 0 and of radius ϵ to ensure that the constraint $||\eta^{'}||_2 \leq \epsilon$ is satisfied. The attack is summarized in Algorithm 2. As can be seen, the attack uses a similar while loop as in DeepFool and the projection function introduced in the PGD attack.

C. System Architecture

The Robot Operating System (ROS) [33] is the most popular software framework in robotic research and applications. One example of an attack that injects malicious data into a running ROS application is the Stealth Publisher Attack [34]. We exploit the same vulnerability to inject adversarial perturbations into a running end-to-end driving ROS application.

We design an adversarial system to attack the end-to-end autonomous driving system (See Fig. 2). The system consists of three key components: the simulator, the server, and the Web User Interface (UI). The simulator publishes the image captured by the front camera to the server. Meanwhile, it accepts steering commands from the server to manipulate the vehicle. The modular design pattern makes it possible to conveniently replace the simulator with a real Turtlebot without breaking the whole system. The server receives input images from the simulator via WebSocket connections and then sends back the control commands. Meanwhile, it receives attack commands from the web browser and then injects the adversarial perturbation into the input image. The end-to-end driving model is deployed on the server as well. We use a website as a front-end where the attacker can monitor the status of the simulator and choose different attacks. The experimental results are presented in the next section.

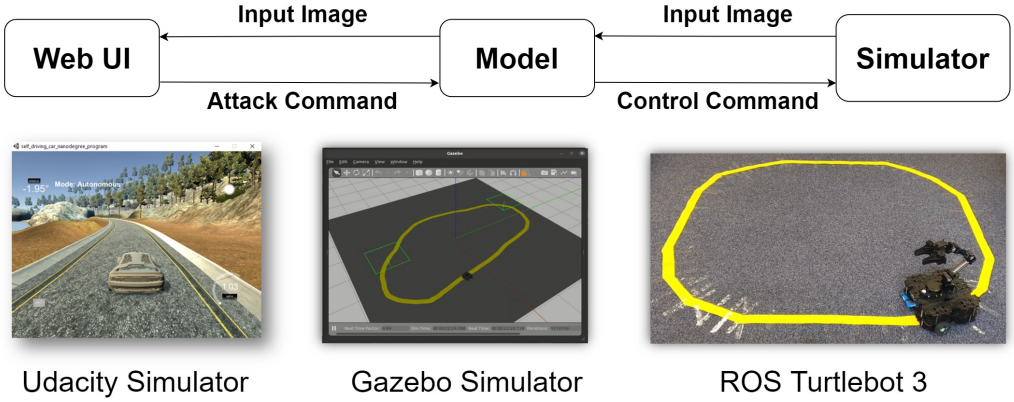


Fig. 2: The architecture of the Adversarial Driving System. We tested our attacks in three environments: the Udacity Simulator, the ROS Gazebo Simulator, and a real Turtlebot 3.

V. Experimental Results

This section first describes the training of the driving systems. Following this, we describe the performance of our proposed image-specific and image-agnostic attacks.

A. Model Training

Our objective is to implement a real-time online attack against an end-to-end imitation learning model. Since it is risky to perform online attacks against real-world driving systems, we tested our attacks in self-driving simulators.

The target imitation learning models were trained from human driving records. In total, we collected 32k images of human driving records in our test environments: the Udacity Simulator (8k), the Gazebo Simulator (12k), and a real Turtlebot 3 (12k). We then trained one end-to-end driving model for each individual environment. The structure of the driving model is detailed in Table I. Our experiments showed that all three models were vulnerable to adversarial attacks.

In the following sections, we use the data from the Udacity Simulator to analyze the attack since experiments in this environment are easier to reproduce and examine than using a Turtlebot for other researchers. The experiments conducted using the Gazebo simulator are illustrated in the demo video.

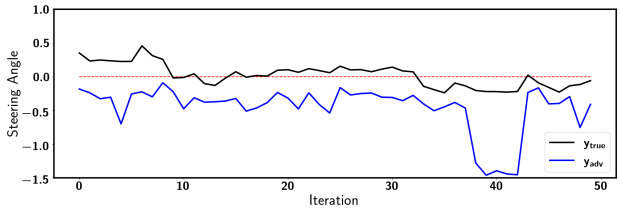
Layer	Output Shape	Parameters
Input	(None, 160, 320, 3)	0
Conv2D	(None, 78, 158, 24)	1824
Conv2D	(None, 37, 77, 36)	21636
Conv2D	(None, 17, 37, 48)	43248
Conv2D	(None, 15, 35, 64)	27712
Conv2D	(None, 13, 13, 24)	36928
Dropout	(None, 13, 13, 24)	0
Flatten	(None, 27456)	0
Dense	(None, 100)	2745700
Dense	(None, 50)	5050
Dense	(None, 10)	510
Dense	(None, 1)	11

TABLE I: The structure of the end-to-end driving model.

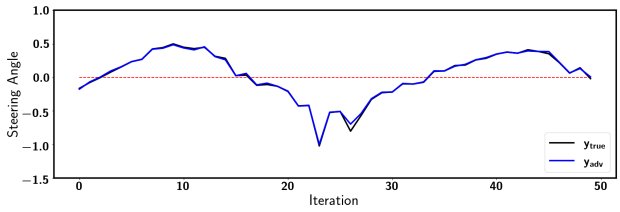
B. The Image-Specific Attack

To begin with, we demonstrate that applying random noise to the end-to-end driving model only results in very small deviations. The parameter ϵ is used to ensure that the total disturbance of the random noise is the same as from the image-specific attack. Specifically, note that the image-specific attack adds or subtracts ϵ from each pixel based on the sign of the gradient. Likewise, we construct a random noise perturbation that randomly adds or subtracts ϵ from each pixel.

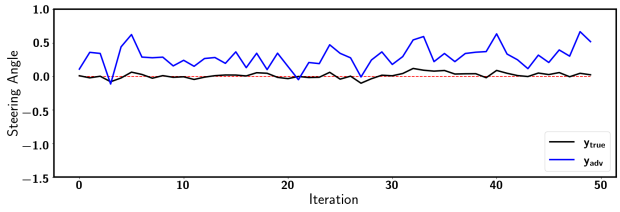
In Fig. 3, we applied three different attacks that are of the same strength. Once under the image-specific attack, the vehicle drove off the road in several seconds. The image-specific left attack deviates the vehicle to the left by decreasing the steering angle, thus the y_{adv} is smaller than y_{true} in Fig. 3a. On the other hand, the image-specific right attack deviates the vehicle to the right by increasing the steering angle, thus the y_{adv} is greater than y_{true} in Fig. 3c. The random noise perturbation barely deviates y_{adv} from y_{true}, indicating that it has little effect on the driving model.



(a) The image-specific left attack decreases the steering angle.



(b) The random noise perturbation barely deviates y_{adv} from y_{true}.



(c) The image-specific right attack increases the steering angle.

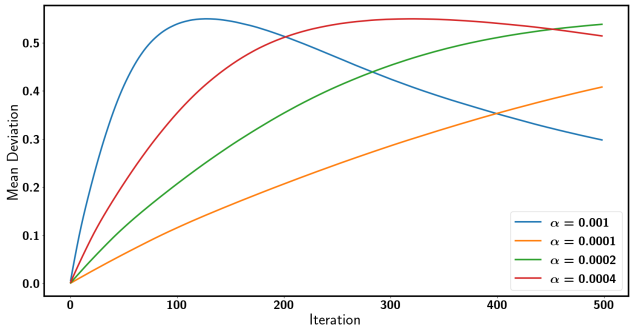
Fig. 3: The image-specific attack and random noise with the same strength ($\epsilon = 1$).

The image-specific attack achieved 20-30 FPS on an Intel i7-8665U CPU and 600-700 FPS on an NVIDIA RTX 2080Ti GPU. Since the CPU and GPU are also utilized for Udacity Simulator, the attack performance varies depending on the hardware temperature.

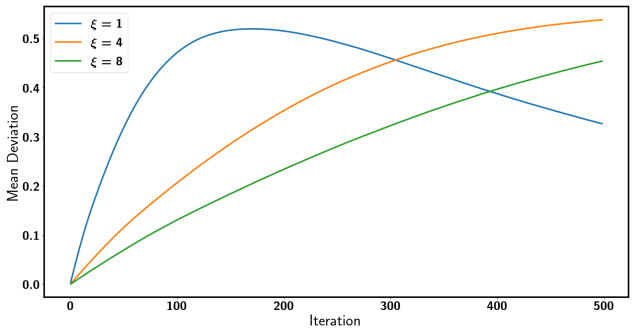
Further, we measured the MAD of the steering angle over 800 attacks. The results are shown in Table II. As can be seen, even the weakest image-specific attack ($\epsilon = 0.1$) is much stronger than the strongest random noise attack ($\epsilon = 8$). When $\epsilon = 4$ and $\epsilon = 8$, we can even deviate the steering angle outside of the range $[-1, 1]$. In other words, the image-specific attack is very strong. However, its weakness is that it needs to calculate the gradients of each individual input image. In a real-world scenario, we may not have access to the input image and gradients. Thus, we propose the image-agnostic attack that trains the perturbation using driving records and does not need access to the input and gradients during the deployment.

Attack Strength	Random Noise Attack	Image-Specific Attack
$\epsilon = 0.1$	0.0002	0.1448
$\epsilon = 1$	0.0020	0.4779
$\epsilon = 2$	0.0048	0.7329
$\epsilon = 4$	0.0150	1.4895
$\epsilon = 8$	0.0278	2.4469

TABLE II: The mean absolute deviation of the steering angle over 800 image-specific attacks.



(a) Different α with fixed $\epsilon = 1, \xi = 4$.



(b) Different ξ with fixed $\alpha = 0.002$

Fig. 4: The mean deviation of the steering angle during the training process with different hyperparameters.

C. The Image-Agnostic Attack

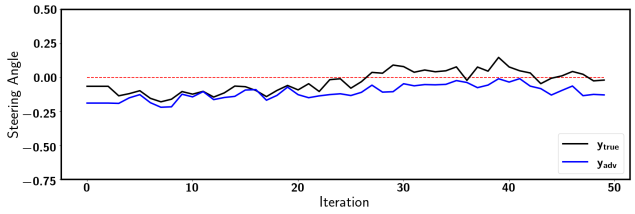
In similarity with the image-specific attack, the strength of the image-agnostic attack was also compared with a random noise attack. The results are shown in Table III. Though the image-agnostic attack is weaker than the image-specific attack, it is still stronger than the random noise attack.

Attack Strength	Random Noise Attack	Image-Agnostic Attack
$\epsilon = 0.1$	0.0002	0.0373
$\epsilon = 1$	0.0020	0.1109
$\epsilon = 2$	0.0048	0.1294
$\epsilon = 4$	0.0150	0.1131
$\epsilon = 8$	0.0278	0.1275

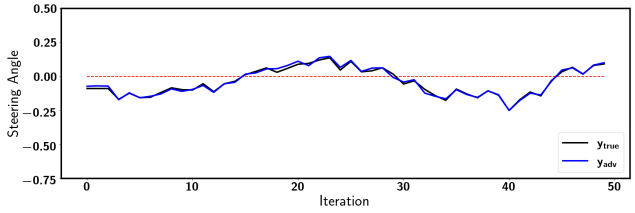
TABLE III: The mean absolute deviation of the steering angle over 800 image-agnostic attacks ($\alpha = 0.002,\ \xi = 4, n = 500$).

As seen in Table III, the strength of the image-agnostic attack does not improve after $\epsilon > 2$. This is due to the limited generalizability of the perturbation. Increasing the strength of the attack further may increase the model output for some inputs but may equally well decrease the model for other inputs. Therefore, increasing ϵ further adds more variation to the model prediction while the MAD remains stable.

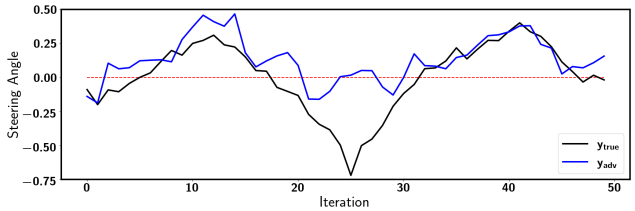
We also investigated the effect of the learning rate α and the step size ξ on the training process (See Fig. 4). The learning rate α controls the variation of the perturbation during the whole iteration. We tested different α with fixed parameters



(a) The image-agnostic Left Attack decreases the model output ($y_{adv} < 0$), making it difficult to turn right. ($\epsilon = 1$)



(b) The random noises barely deviates y_{adv} from y_{true}.



(c) The image-agnostic Right Attack increases the model output ($y_{adv} > 0$), making it difficult to turn left. ($\epsilon = 1$)

Fig. 5: The Image-Agnostic attack ($\alpha = 0.002$, $\xi = 4$, $n = 500$) and random noises with the same strength ($\epsilon = 1$).

$\epsilon = 1$ and $\xi = 4$. As α increases, the mean deviation increases faster. However, the iteration process also becomes more variable, and the mean deviation decreases after 100 steps when $\alpha > 0.01$.

The step size ξ decides how fast the perturbation is updated to change the model output to the desired direction for each input image x. A smaller ξ makes the update towards the target direction more steady, but the iteration takes a longer time. A larger ξ can change the direction of the model output in a single step, but the perturbation may not generalize well to other inputs.

As illustrated in Fig. 5, using the parameters $\alpha = 0.0002$ and $\xi = 4$ enabled us to generate image-agnostic perturbations at $\epsilon = 1$ that are comparable in performance with the image-agnostic attack at $\epsilon = 0.1$. While the image-agnostic is not as strong as the image-specific attack, the image-agnostic attack makes the vehicle difficult to control at sharp corners (this is illustrated in the demo video), which could lead to incidents at some critical points.

In addition, the image-agnostic attack applies the same perturbation to all frames. Thus, the deployment of the image-agnostic attack is much more computationally efficient than the image-specific attack.

VI. Conclusions

This paper has demonstrated that it is possible to attack an end-to-end driving model in real-time. We devise a strong image-specific attack and a stealthy image-agnostic attack. Though the mean absolute deviation of the image-agnostic attack is smaller than the image-specific attack, both attacks are more effective than random noise attacks. The image-agnostic attack deviates the vehicle outside of the lane after just a few seconds, while the image-agnostic attack could cause incidents at sharp corners. These results provide new evidence of the vulnerability of safety-critical robotic applications.

References

[1] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[2] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, 2017, pp. 1–16.

[3] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "Airsim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, 2017.

[4] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[5] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A survey of autonomous driving: Common practices and emerging technologies," *IEEE Access*, vol. 8, pp. 58 443–58 469, 2020.

[6] D. Chen and P. Krähenbühl, "Learning from all vehicles," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 17 222–17 231.

[7] R. Chopra and S. S. Roy, "End-to-end reinforcement learning for self-driving car," in *Advanced Computing and Intelligent Engineering*, 2020, pp. 53–61.

[8] Ó. Pérez-Gil, R. Barea, E. López-Guillén, L. M. Bergasa, C. Gomez-Huelamo, R. Gutiérrez, and A. Diaz-Diaz, "Deep reinforcement learning based control for autonomous vehicles in carla," *Multimedia Tools and Applications*, vol. 81, no. 3, pp. 3553–3576, 2022.

[9] J. Kabudian, M. Meybodi, and M. Homayounpour, "Applying continuous action reinforcement learning automata (carla) to global training of hidden markov models," in *International Conference on Information Technology: Coding and Computing (ITCC)*, vol. 2, 2004, pp. 638–642.

[10] Y. Jaafra, J. L. Laurent, A. Deruyver, and M. S. Naceur, "Seeking for robustness in reinforcement learning: application on carla simulator," 2019.

[11] K. Chitta, A. Prakash, and A. Geiger, "Neat: Neural attention fields for end-to-end autonomous driving," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021, pp. 15 793–15 803.

[12] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad, "A survey of end-to-end driving: Architectures and training methods," *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

[13] A. Prakash, K. Chitta, and A. Geiger, "Multi-modal fusion transformer for end-to-end autonomous driving," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 7077–7087.

[14] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, "Transfuser: Imitation with transformer-based sensor fusion for autonomous driving," *Pattern Analysis and Machine Intelligence (PAMI)*, 2022.

[15] P. Wu, X. Jia, L. Chen, J. Yan, H. Li, and Y. Qiao, "Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline," *arXiv preprint arXiv:2206.08129*, 2022.

[16] D. A. Pomerleau, "Alvinn: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems*, vol. 1, 1989.

[17] U. Muller, J. Ben, E. Cosatto, B. Flepp, and Y. Cun, "Off-road obstacle avoidance through end-to-end learning," in *Advances in Neural Information Processing Systems*, vol. 18, 2006.

[18] Y. Li, M. Cheng, C.-J. Hsieh, and T. C. Lee, "A review of adversarial attack and defense for classification methods," *The American Statistician*, vol. 76, no. 4, pp. 329–345, 2022.

[19] J. Zhang, Y. Lou, J. Wang, K. Wu, K. Lu, and X. Jia, "Evaluating adversarial attacks on driving safety in vision-based autonomous vehicles," *IEEE Internet of Things Journal*, vol. 9, no. 5, pp. 3443–3456, 2021.

[20] A. Boloor, X. He, C. Gill, Y. Vorobeychik, and X. Zhang, "Simple physical adversarial examples against end-to-end autonomous driving models," in *Proceedings of the IEEE International Conference on Embedded Software and Systems (ICESS)*, 2019, pp. 1–7.

[21] Z. U. Abideen, M. A. Bute, S. Khalid, I. Ahmad, and R. Amin, "A3d: Physical adversarial attack on visual perception module of self-driving cars," 2022.

[22] S. Villar, D. W. Hogg, N. Huang, Z. Martin, S. Wang, and G. Scanlon, "Adversarial attacks against linear and deep-learning regressions in astronomy," in *Proceedings of the 1st Annual Conference on Mathematical and Scientific Machine Learning*, 2019.

[23] A. T. Nguyen and E. Raff, "Adversarial attacks, regression, and numerical stability regularization," *arXiv preprint arXiv:1812.02885*, 2018.

[24] A. Boloor, K. Garimella, X. He, C. Gill, Y. Vorobeychik, and X. Zhang, "Attacking vision-based perception in end-to-end autonomous driving models," *Journal of Systems Architecture*, vol. 110, p. 101766, 2020.

[25] Y. Deng, X. Zheng, T. Zhang, C. Chen, G. Lou, and M. Kim, "An analysis of adversarial attacks and defenses on autonomous driving models," in *IEEE International Conference on Pervasive Computing and Communications (PerCom)*, 2020, pp. 1–10.

[26] K. Ren, T. Zheng, Z. Qin, and X. Liu, "Adversarial attacks and defenses in deep learning," *Engineering*, vol. 6, no. 3, pp. 346–360, 2020.

[27] K.-H. Chow, L. Liu, M. Loper, J. Bae, M. Emre Gursoy, S. Truex, W. Wei, and Y. Wu, "Adversarial objectness gradient attacks in real-time object detection systems," in *IEEE International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications*, 2020, pp. 263–272.

[28] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, "Square attack: a query-efficient black-box adversarial attack via random search," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020, pp. 484–501.

[29] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1765–1773.

[30] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2574–2582.

[31] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," *arXiv preprint arXiv:1706.06083*, 2017.

[32] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.

[33] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[34] B. Dieber, R. White, S. Taurer, B. Breiling, G. Caiazza, H. Christensen, and A. Cortesi, "Penetration testing ros," in *Robot Operating System (ROS)*. Springer, 2020, pp. 183–225.